

Chapter 0

An Overview and Roadmap

Sushil K. Prasad¹, Alan Sussman², Neena Thota³, Ramachandran Vaidyanathan⁴ and
Charles Weems³

¹University of Texas, San Antonio⁵

²University of Maryland, College Park

³University of Massachusetts, Amherst

⁴Louisiana State University, Baton Rouge

⁵Lead Institution; Contact email: sushil.prasad@gmail.com

Abstract (Sushil)

Modern software is increasingly parallel, distributed, networked, event-driven, graphical, API-based, and data-intensive. However, introductory computing courses still commonly begin with a sequential-first model of computation which does not fully reflect the systems students encounter today. Students regularly use applications that depend on multicore processors, remote services, asynchronous events, large data streams, and libraries that hide complex computation. Yet many students do not learn these ideas formally until advanced electives, if at all. This gap limits their preparation for later study and work in software engineering, artificial intelligence, cybersecurity, high-performance computing, computational science and engineering, and other areas of computing and applications.

This volume responds with a practical adoption guide for the introductory computing course sequence for instructors, course coordinators, departments, and academic leaders. Rather than only arguing that introductory courses should be modernized, it provides classroom-tested, modular course materials that can be adopted at different scales. The goal is not to turn introductory computing students into parallel programmers, but to help them develop an early conceptual model in which computation may involve multiple workers, remote data, performance tradeoffs, and coordination among interacting components, all the while ensuring that the basic CS1 learning outcomes are preserved.

This work results from an NSF-funded multi-institutional project to create modern CS1 and CS2 course exemplars infused with parallel and distributed computing (PDC) concepts that can serve as easily adoptable models.¹ To enable this vision, the project began with two initial course exemplars developed in different settings: a Java-based sequence at a small liberal arts college and a C/C++-based sequence in a medium-sized computer science department in an engineering-college setting. These exemplars were then adopted, adapted, and evaluated by six carefully recruited institutions ranging from a community college to private undergraduate institutions and an R1 university. This diversity is central to the project: no single model fits all languages, calendars, class sizes, student populations, or instructor backgrounds. This project thus emphasizes multiple pathways for PDC infusion.

This first release focuses on the CS1 portion of the e-book and is being made available early to support possible Fall-term adoption. It includes eight institutional CS1 course exemplars that have been classroom tested and evaluated, together with shared activities and *substantial sets of instructional resources*, which would be periodically curated. Across these chapters, readers will find a range of approaches: short conceptual activities requiring little or no programming background; unplugged exercises that model processors, data partitioning, speedup, load imbalance, and communication; visualizations and simulations that make abstract PDC concepts visible; lightweight programming assignments that connect PDC ideas to loops, arrays, sorting, searching, APIs, and event handling; and assessment instruments that capture student background, self-reported experience, engagement, and learning evidence. Together, the chapters illustrate that PDC can be introduced through existing CS1 topics rather than treated only as additional content.

This chapter serves as a roadmap for the volume. Readers may navigate the material by programming language, activity type, class size, instructional time, institutional context, or desired level of adoption. An instructor seeking a single activity can begin with the distilled

¹NSF Award #2321015

set of common activities described in chapter 1 and then consult institutional examples for implementation details in chapters 2-9, each with their comprehensive instructional material ready to download and adopt. An instructor seeking a module or course-level infusion can compare the institutional chapters, course schedules, assessments, and lessons learned. Some consultation from project personnel will be available to help adopters select and adapt materials. CS2 chapters are expected soon for subsequent-term adoption. Additional instructor training workshops will be organized through a new NSF award to further infuse AI and Big Data, together with PDC, into CS1, CS2, and Computer Systems courses by the CDER center.²

How to Read this Volume: Depending on your primary objective, we provide some initial short cuts to relevant parts of the volume. Nevertheless, the volume in its entirety provides a perspective and depth that may not be available from a selective reading.

- Instructor seeking to include new material: Start from Section 0.5 and then move to relevant chapters as indicated in that section.
- Researcher looking for pedagogical approaches and research results start from Section 0.3.
- Those looking to understand the basis and impact of this project continue to read this chapter.

²<https://cdercenter.org/>

0.1 Introduction

The CDER center organized a series of stakeholder workshops during 2020-23 supported by a National Science Foundation (NSF) CyberTraining Institute Conceptualization grant³ that examined the systemic barriers to broader PDC adoption, including the continued dominance of a sequential model of computation in early courses. Multiple employer stakeholders including national labs and agencies reported that computer science and engineering students are not graduating with a sufficient grasp of the software development process for the modern computing ecosystem, which relies on parallelism, distribution, asynchrony, scaling, integration across disparate libraries and data sources, test-driven design, and pervasive concerns for security. The lack of preparation in parallel and distributed computing (PDC) is rooted in an obsolete algorithmic model of computing (sequential, synchronous, with text-based I/O) that instruction relies upon, from introductory programming to data structures, algorithms, and software engineering. Once this model is entrenched, students may only encounter PDC in an *ad hoc* manner via electives. Education stakeholders said that changing the model is nearly impossible because of a lack of exemplars of how introductory courses can be modernized, few teaching materials, a lack of instructor training in modern computing, and little evidence of how changing the model benefits students. Perhaps the greatest impediment is that academic stakeholders find it difficult to visualize what a different approach would look like, and most feel it is not possible to make such a change. What we heard repeatedly is that "someone else needs to go first."

Project Overview

This volume is a result of our followup work on a NSF-funded multi-institutional collaborative project "Modern Course Exemplars infused with Parallel and Distributed Computing for the Introductory Computing Course Sequence (2023–2027)." This project adopts a "go first" effort to create exemplars of first year course sequence (typically CS1 and CS2) changes and accompanying materials, while demonstrating their effectiveness and transferability. The project employed two teams to develop these exemplars in two distinct contexts, and then trained a cohort of instructors at six additional institutions to implement them, along with instrumenting their interventions so that student outcomes can be documented. A backbone team helped coordinate the various moving parts of the project. Figure 1 captures the overall project activities and timeline (Year 1 being 2023). The result is a demonstration of the potential for modernizing the computing model that shapes how first year computing students think computationally, enabling them to conceptualize how software can make use of parallelism and distribution.

Along the way, many opportunities and challenges were identified. Are concepts in PDC too advanced at the early course level? On this we found *unplugged activities* which entail no programming, but underscore key concepts, were very effective. Is there room for introducing new material in an already crowded course? We found it was possible to teach traditional concepts with a different focus; for example, a lesson on loops could be augmented (with a small effort) to include a *Flag Maker activity* wherein students act as processors coloring

³NSF Institute Conceptualization Award #2002649

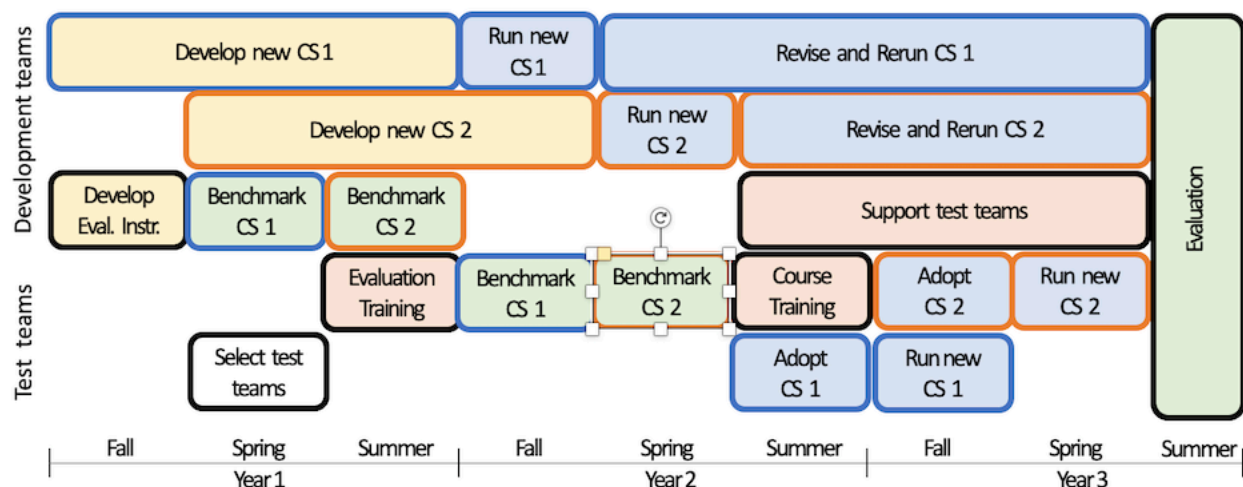


Figure 1: Project activities and Timeline. Box fill colors categorize activities as development (yellow), evaluation (light green), training and support (peach), and running the new courses (purple).

flag-grid cells in sequential and parallel scenarios.

Key Contributions of this Volume: This volume captures the significant findings, observations and deliverables from the project described above. Specifically,

- It describes, in eight chapters, various ways of infusing PDC in CS1. These chapters are complete with instructional material, classroom based evaluation, and adoption and adaptation experiences.
- It distills a set of popular unplugged and programming activities alongside infusion methods challenges faced into Chapter 1.
- It provides a bird-eye view of the project activities that led to this e-book, including team structures, methodology employed, learning community formation, a road map for readers, and the backdrop of prior work (this chapter).

Chapter organization: The remainder of this chapter is organized as follows. Section 0.2 describes the project focus, the development and testing teams, and the role of the backbone team. Section 0.3 explains the PDC topics selected for infusion, the recruitment of testing teams, the collaboration process, and the evaluation methods used across institutions. Section 0.4 discusses faculty learning community formation and sustainability. Section 0.5 provides an adopter-oriented roadmap for navigating the common activity chapter, the institutional CS1 chapters, appendices, and repositories. The appendix situates this project within the broader CDER center activities, including IEEE-CS TCPP curriculum, early-adopter, training, and dissemination efforts, that shaped the work reported in this volume.

0.2 Project Focus and Team Structures

As noted above the project targeted the CS1, CS2 sequence of a typical Computer Science program for the introduction of PDC concepts. For both courses data was obtained both from a baseline offering (without any interventions) and from an offering after the PDC ideas were introduced. The teams were also introduced to methods and instruments for evaluating the efficacy of the interventions. In general CS1 ideas were developed first and experience with CS1 informed our approach to CS2. The main ideas introduced can be summarized as follows:

- Data parallelism and its applications
- Distributed data access
- Event driven processing

The project was implemented by three sets of teams that worked closely together.

Development Teams: These teams form the frontline in the implementation and testing of new interventions in their first computing courses that introduce PDC concepts. The Development Teams, with faculty from two different types of institutions and using two different programming languages, each created exemplars for their CS1 and CS2 computing courses, such that the second course builds on the concepts introduced in the first. These teams coordinated their efforts so that the same PDC concepts were covered, although in ways that are specific to each of their contexts. After a trial run of the new material by the development teams, they worked with the Testing Teams (see below) to transfer their exemplars to six other institutions. The experiences of the Testing Teams were then used to improve the work of both sets of teams.

There were two Development Teams, one at Tennessee Technological University and one at Knox College. Details of these teams are listed here:

1. Tennessee Technological University (TNTECH) is a public research university in Cookeville, Tennessee that offers undergraduate and graduate degrees in Computer Science (the department associated with this project). The institution operates on a semester calendar.
 - Team members: April Crockett, Gerald Gannod
 - Language used: C++
2. Knox College (Knox) is a private liberal arts college in Galesburg, Illinois that offers an undergraduate degree in Computer Science (again the department associated with this project). The institution operates on a quarter calendar.
 - Team members: David Bunde, Jaime Spacco
 - Language used: Java

Testing Teams: These teams worked to transfer the exemplars introduced by the Development Teams into their own courses, in the process identifying opportunities and

challenges. The input from the Testing Teams was a key element in improving the teaching materials developed by the Development Teams.

Six Testing Teams described below worked on the project.

1. Casper College (Casper) is a public community college in Casper, Wyoming.
 - Team member: Charlotte Gruner
 - Programming language: C++, Java, Python
2. Hawaii Pacific University (HPU) is a private university in Honolulu, Hawaii.
 - Team member: Mary Smith
 - Programming language: Java
3. Montclair State University (MSU) is a public university in Montclair, New Jersey.
 - Team members: Jiayin Wang and Michelle Zhu (currently at Kennesaw State University)
4. University of Southern Indiana (USI) is a public university in Evansville, Indiana.
 - Programming language: Java
 - Team member: Srishti Srivastava
 - Programming language: Java
5. University of Nebraska, Lincoln (UNL) is a public land-grant university in Lincoln, Nebraska.
 - Team member: Chris Bourke
 - Programming language: C, Java
6. Webster University (Webster) is a private university with a main campus in Webster Groves, Missouri.
 - Team members: Xiaoyuan Suo and Peter Maher
 - Programming language: C++

Backbone Team: This team coordinated the project, organizing training workshops and biweekly meetings, facilitating discussion, and providing domain expertise in PDC. The Backbone Team members are

- Sushil K. Prasad, University of Texas, San Antonio
- Alan Sussman, University of Maryland, College Park
- Neena Thota, University of Massachusetts, Amherst
- Ramachandran Vaidyanathan, Louisiana State University, Baton Rouge
- Charles Weems, University of Massachusetts, Amherst

0.3 Methodology Employed

0.3.1 PDC Topics Chosen: Data Parallelism, Distributed Data Access, and Event Driven Processing

When instructors hear about incorporating PDC into CS1 and CS2, there is a tendency to think that the subjects are too advanced, or that they require too much infrastructure.

However, because PDC has become so fundamental to practice, all modern programming languages include support for PDC, either through standard libraries or language constructs. These can be employed with cognitive loads similar to using common object classes, such as vectors, lists, sets, maps, and dictionaries.

It is important to understand that, in early courses, the goal in experiencing PDC is to open the student’s mind to an appreciation that algorithmic problem solving can encompass concepts of parallelism and distribution. There is no expectation that students will gain a higher level of skill in implementing these concepts than they do with any other language features or classes employed at this level.

Another thought that often arises among instructors is that there is no space in these courses to add material. But our analysis of widely used textbooks and syllabi from select institutions showed that there is a great deal of material that is dealing with limitations arising from teaching around a computing model that dates to the 1980s.

For example, in CS1, it is typical to see branching and looping focused on parsing and error checking of text typed on a console or read from a text file, that is being stored into primitive types. Formatting console text output (e.g., printing tables with columns and headings) is another common pattern.

But modern user I/O APIs include data checking, and file I/O APIs enable writing and reading serialized objects, even with remote repositories (illustrating distribution of data). They thus avoid the impression that I/O is entirely about streams of chars on the local machine, which is an entirely foreign paradigm to internet-native students whose prior experience of computing is via graphical interfaces. Such interfaces also exemplify event-driven processing, which illustrates many basic ideas behind control parallelism.

In existing CS2 courses, considerable effort may be spent on implementing linked structures from primitive types, including building APIs that exhibit weak robustness, and using fixed algorithms where the efficiency doesn’t scale with data volume. Modern container and collection classes can instead serve as examples of good design and provide opportunities to explore their underlying algorithms. Thus, there are indeed possibilities for making space to add PDC in these courses, when they are updated to use more modern tools.

What will also be seen in these chapters is that, in many cases, PDC concepts can be overlaid on existing topics with a modest investment of time. For example, if a recursive mergesort is already taught, it is easy to see that parallelism can be exploited by making the first few levels of calls spawn separate threads. Another approach is to introduce concepts of parallelism or distribution through unplugged activities, and then have students relate those to provided or scaffolded code that enables them to see the benefits of applying the concepts.

The development teams thus created examples of courses where PDC concepts related to data parallelism, distributed data access, and event-driven processing are incorporated as a natural aspect of learning to solve problems algorithmically, which they then tested with their own students before presenting them to the testing teams for adoption.

0.3.2 Testing team recruitment

A primary component of this project was to ensure that course exemplars were tested and adopted in different contexts. Toward that end we successfully recruited six testing

teams, with institutions ranging from a community college to an MSI to an R1 university, both public and private, and geographically widely dispersed. We recruited teams using our CDER center mailing list, and listserves such as TCPP-announce and SIGCSE, plus promotion at our Edu* workshops, as well as our CDER booth at SIGCSE'24. We also held a webinar for the potential recruits, laying out the project goals and significance, application requirements, and expected outcomes and support available.

This groundwork resulted in good quality applications which were evaluated first on the basis of whether the applicant was teaching one of the target courses, whether their institution was open to modifying their CS1 and CS2 courses, and whether they were using one of the two target languages (Java or C++). A secondary basis for the evaluation was to balance the number of testing teams for each language and to ensure that the set of teams included a variety of institution types and sizes. We selected six teams comprising one or two faculty per team as described in Section 0.2.

It turned out that each of the teams had at least one person who had participated in one of our earlier training workshops. We held a kickoff meeting in May of that year laying out the project goals, timelines, and testing team responsibilities. Because the earlier training had included the IRB application process and evaluation methods, it was only necessary to review these. The development teams also provided a status report on their course development efforts. This was followed by a series of biweekly meetings where the teams explained their institutional contexts, described introductory course syllabi, and worked on plans for IRB approvals.

These early meetings lead to in an in-person summer workshop where the project personnel assembled. Because the testing teams all had prior training (and some had already developed and tested PDC course interventions as a result), they were able to hit the ground running, and the focus quickly shifted to brainstorming pedagogy and evaluation methodology.

0.3.3 Collaboration among Development and Testing teams, and support of Backbone team

Following the first summer meeting, the teams met regularly online. The CDER backbone team met weekly to plan the meetings with the other teams, and also to work on related activities by the center, such as our series of PDC education workshops associated with SC, IPDPS, and HiPC, tutorials and a booth at SIGCSE, development of a new monograph presenting examples of infusing PDC into courses, a journal special issue, planning the summer project meeting, and recruiting new trainees for a separate summer workshop.

The backbone and development teams met weekly to work on course development, review reports from the testing teams, and plan our meetings with them. All three teams met bi-weekly, starting in the second year, as the testing teams were working on their IRBs and implementations. In the third year efforts shifted to running the new courses and gathering data. Many of the team members would get together at conferences to discuss the project, and additional summer meetings were held at the end of the second and third years.

0.3.4 Exemplar Creation and Testing by the Development Teams

TTU CS1 and CS2 Exemplar Development (C++)

Tennessee Tech students begin the semester with a basic understanding of sequential vs. parallel computing through an *unplugged penny sorting* group activity. The activity has several timed phases to show speedup with parallel phases as well as load balancing.

Students are introduced to topics related to distributed computing in the functions module. They are taught when a problem solution requires interaction with a *remote service* and they have a lab programming assignment where they pull and display data from a real-world, live, remote service.

Students are introduced to the range-based for loop (for-each loop) in the arrays/vectors module. Although it doesn't execute in parallel, the for-each construct exemplified dependence-free processing of a collection, which can be seen as a candidate for parallelism. They are also given an introduction to *data parallelism* and are taught how to identify algorithmic solutions that exploit it. Students have a lab assignment where they demonstrate data parallelism with a vector. Instruction has been modernized in this module to spend more time discussing the Vectors and Arrays classes in the C++ standard template library. The lab assignment is paired with an unplugged activity focusing on data parallelism. This unplugged activity is a *flag-maker activity* where students use markers to color in pixels of the flag in certain orders to demonstrate the use of for-each loops. This activity has several phases to demonstrate the following concepts: sequential vs. parallel programming, speedup, scalability, synchronization, underutilized resources, load balancing, pipelining, contention, and race conditions.

Near the end of the semester when object-oriented programming with classes and objects are introduced, students are introduced to *event handling* and taught when an algorithmic solution can benefit from the use of multiple threads.

For CS2, TTU modified a module where the performance of sorting algorithms was compared by including the use of *OpenMP* to parallelize a bubble sort (as an odd-even transposition sort), and a recursive merge sort. Because OpenMP makes this relatively easy, via the insertion of a few pragmas, it adds little to the time or cognitive load for the module, but enables students to clearly see the benefit of parallelism for sorting large vectors.

Knox CS1 and CS2 Exemplar Development (Java)

Knox College developed several modules for CS1. *Flag Maker*: Knox updated a version of an existing programming assignment in which students draw flags pixel by pixel using nested loops. It was changed to use JavaFX (previously in Swing) and to add clicker questions about the order in which pixel colors are assigned and which operations are independent. (The simplest way to draw some flags is in layers, which introduces dependencies if the drawing is done in parallel.) An unplugged activity was also added (students using markers) to this module.

Testing "mystery code": A new tool for testing that lets students just give input/output pairs rather than explicitly using JUnit or similar frameworks. The assignment gives a series of methods and the students must figure out what they do by giving inputs and observing the outputs. This activity teaches lessons about confirmation bias, using distinct test cases,

and verifying hypotheses with test cases. It is motivated as a shift toward testing and understanding code rather than writing it from scratch since generative AI can assist with writing.

Events and using a framework: Using the Greenfoot IDE as a framework for interactive code; code is attached to sprites and called by events from the system rather than from a main method. Students are introduced to the event-driven programming model and provided some examples, after which they created a variety of games (open-ended assignments).

Interactive server: Knox created several versions of this lab, which has students interact with a provided server to illustrate distributed data. The original version stored calendar dates and was not sufficiently interesting. For the next offering, students ran instructions to create 3D objects using a generalization of Logo (turtle graphics) to a Minecraft world. A version of r/place was also created, where participants can place pixels at a throttled rate to collaboratively create an image.

For CS2, Knox created a module introducing the *fork-join framework* for parallel computing. This gives students practice writing recursive code. Analysis is done in terms of work and span following the treatment of CLRS in a way that feels similar to the rest of the course. For programming assignments, students parallelize a graphical heat diffusion simulation (producing a visibly-faster program) and also implement a variety of parallel reduction operations.

0.3.5 Adaptation and Adoption by Testing Teams

At our first summer meeting, the CS1 exemplars from the development teams were presented, and then collaboration with the testing teams led to significant refinements to improve transferability and incorporate additional implementation ideas. During the following year, testing teams applied for IRB approval, and specific research questions were developed for each institution’s study, with a focus on coordinated data collection for cross-team analysis. At the second summer meeting, data from the baseline runs, and some initial experimental CS1 runs by the testing teams was presented and then the focus turned to similar collaboration on the CS2 exemplars that had been developed, which were then tested in year three. The chapters in this book illustrate the outcomes of this process, where teams report how they adapted the exemplars to their unique contexts and existing course syllabi.

0.3.6 Evaluation Methods

We developed the following instruments that were administered during the course of the project.

Students:

1. Pre-course and post-course surveys assessing experience and attitudes toward PDC given to students by the development and testing team instructors.
2. Lab assignment and programming assignment surveys for students to determine how much time the students spend on the assignments, which days they worked on the assignments, what they learned from the assignment, the most challenging aspect of

the assignment, if they still have questions about the assignment, and the extent to which they enjoyed the assignment.

Faculty Instructors - Development and Test Teams

1. Pre-selection surveys with an objective component regarding the test team faculty applicants' background in PDC, current use of PDC in classes, institution, courses, student population, and a subjective component assessing initial attitudes toward incorporation of PDC into lower level classes.
2. Post-selection surveys of the Test Team faculty instructors on the appropriateness and effectiveness of the exemplars, likelihood of incorporation of PDC into other related classes, and initial ideas for the approaches and intervention points for incorporation of the exemplars in their existing courses.
3. Annual surveys(conducted during summer working sessions) of the Test Team faculty instructors to assess the number of students actually taught, degree to which PDC is incorporated, and the degree to which the interventions and evaluations are effective.
4. Post-adoption survey of the Test Team and Development Team faculty instructors to assess likelihood that the interventions will be retained, whether they will be reduced or expanded, and new directions being taken as a result of the experience.

0.4 Faculty Learning Community Formation

Community building: The project offered the participating faculty a sustained opportunity to integrate PDC content in their courses. Their engagement in this faculty learning community (FLC) allowed fruitful multi-institutional collaborations for developing and teaching the content. We can envision the faculty as change agents who amplified the impact of what they learned into widespread knowledge and use impacting a) the multiple cohorts of students who emerged with a better understanding of PDC concepts and b) the colleagues at their colleges and departments who wish to modernize the CS curriculum.

Sustainability: The project created exemplars for the introductory computing course sequence that will enable a broad set of programs to incorporate PDC concepts, and move forward to a 21st century computing paradigm. Our outreach to a set of testing teams set the stage for a sustainable and scalable effort to train a diverse set of instructors at diverse institutions in the benefits of incorporating PDC concepts from the beginning of a student's education in computing. Those testing teams, along with the teams developed the exemplar course sequence and the rest of the group participating in the project, who now serve as a trained cohort of educators who can reach out to their colleagues at other institutions, eventually training many thousands of students in introductory computing courses per year. The course exemplars developed in the project are now available online, via this book and the associated repository, so that adoption can continue.

0.5 Roadmap for Subsequent Chapters (Sushil)

This volume is designed for selective, adoption-oriented reading. Readers do not need to read every chapter before beginning to identify useful material. Instead, they should begin with their adoption goal, use Chapter 1 to understand the common CS1 activity families, and then consult one or more institutional chapters for local implementation details, teaching experience, evidence, and repository resources. The table of contents gives the full chapter sequence; this section provides a shorter guide to what to read first.

0.5.1 Start from Your Adoption Goal

Potential adopters usually approach this volume with different levels of commitment and different time constraints. Some may want one class activity that introduces PDC concepts without changing the course structure. Others may want a short module, a laboratory activity, or a more systematic CS1 infusion. Figure 2 summarizes three common reading paths.

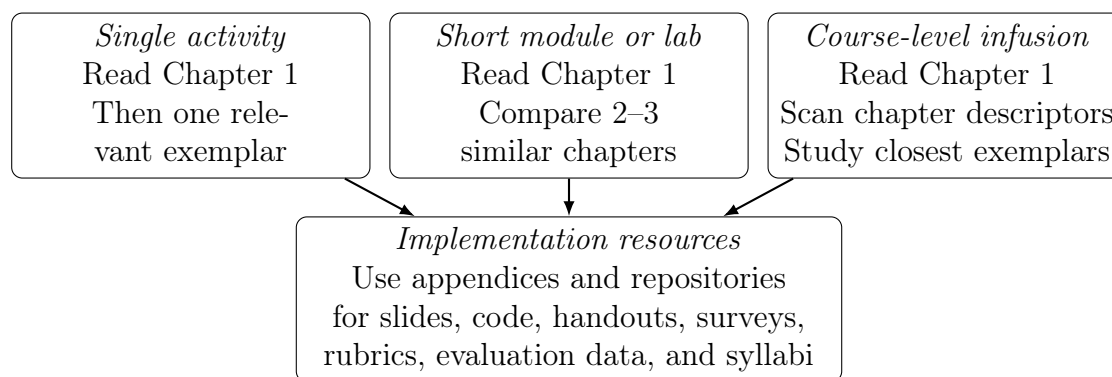


Figure 2: Suggested reading paths by CS1 adoption goal.

For a single class activity, begin with Chapter 1 and look for unplugged, visual, or short discussion-based activities. For a short module or lab, read the common description in Chapter 1 and compare two or three institutional chapters with similar constraints. For a course-level infusion, browse through Chapter 1 and scan the descriptor elements and course-change sections in the institutional chapters, then read the chapters that most closely match the local course language, class size, schedule, and instructional goals.

A useful first-adoption strategy is to choose the simplest version of an activity that fits local constraints. After one offering, instructors can expand the activity, add a plugged component, collect more systematic evidence, or connect the activity to a broader CS1 module.

0.5.2 CS1 Exemplars at a Glance

Table 1 gives a compact view of the eight institutional CS1 exemplars. Readers should treat the table as a starting filter. The most important adoption questions are not only

Table 1: Quick comparison of CS1 institutional exemplars for potential adopters.

Chapter	Institutional context	Language	Primary PDC infusion style	Good starting point for
Ch. 2 TNTECH	Mid-sized, public university; large CS1 lecture/lab setting	C++	Unplugged Penny and Flag Maker activities; OpenMP/data-parallel programming; remote-data/API assignment	Large-section CS1 adoption; programming-linked PDC infusion; multi-semester evidence
Ch. 3 Knox	Small residential liberal arts college; Java-based introductory sequence	Java	Conceptual and programming-oriented CS1 activities adapted to a small-college context	Liberal-arts setting; smaller classes; Java-based adaptation
Ch. 4 USI	Public master's-level institution; introductory object-oriented CS1 course	Java	Unplugged Flag Maker Activity, Unplugged Penny Activity, Plugged-In Flag Maker Activity, Plugged-In Parallel Sort Penny Activity, Plugged-In Greenfoot Activity, and activity-level assessment	Structured activity adoption; evidence-rich unplugged and plugged-in examples
Ch. 5 UNL	Large R1 institution; multiple CS1 sections and computing majors	C, Java	Codeless modules, visual/conceptual activities, remote-data ideas, and scalable conceptual infusion	Low-code pathway; large-institution adaptation; conceptual modules; online classes
Ch. 6 Webster	Private primarily undergraduate institution; small-to-moderate CS1 sections	C++	Animations, visualizations, performance examples, and game/simulation-oriented activities	Visualization-first adoption; low-barrier conceptual entry points
Ch. 7 Casper	Two-year public community college; small cohorts and mixed student pathways	C++, Python	Custom digital-textbook material, unplugged activities, OpenMP, remote data, and physical computing	Community-college setting; small cohorts; varied delivery formats
Ch. 8 HPU	Private undergraduate-serving institution with small class sizes	Java	Flag Maker, Penny Sorting, lightweight plugged/code-review extension, and low-resource activities	Small-scale first adoption; low-preparation unplugged activities
Ch. 9 MSU	Public university; focused CS1 intervention	Java	Flag Maker-centered data-parallelism activity with supporting animations and assessment	Minimal-infusion model; short, focused CS1 intervention

“Which language matches mine?” but also “Which activity scale, preparation level, evidence type, and implementation style match my course?”

0.5.3 Using Chapter 1 with Institutional Chapters

Chapter 1 provides the common entry point for CS1 activities that appear across multiple institutions. It distills recurring activity families, shared PDC concepts, common implementation challenges, and cross-institution lessons learned. Most readers should begin there, especially if looking for activities such as Flag Maker, Penny Search or Penny Sorting, animations, visualizations, remote-data/API work, event-driven examples, or lightweight programming assignments. Figure 3 shows how Chapter 1 is organized.

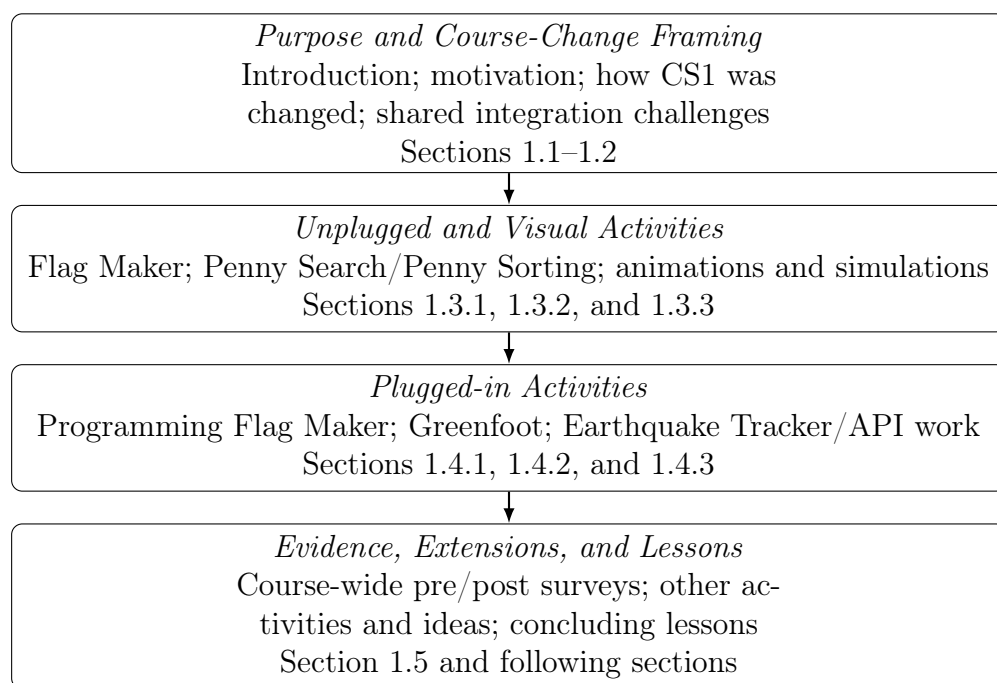


Figure 3: Organization of Chapter 1 – common CS1 activities and adoption-guidance.

Common Institutional Chapter Structure

The institutional chapters follow a common structure so that readers can compare exemplars without first learning a new organization for each chapter. As summarized in Figure 4, each chapter moves from quick screening information, to implementation details, to adoption-ready appendix and repository resources.

The abstract and descriptor elements help adopters quickly decide whether a chapter matches their course setting, language, prerequisites, instructional goals, and desired PDC topics. The chapter body then explains how the course was changed and presents the major activities, including classroom logistics, teaching guidance, challenges, and available evidence. The appendix and repository resources provide the concrete materials needed for adoption, such as syllabi, course calendars, handouts, slides, assignments, starter code, surveys, anonymized data, and analysis summaries. Before using a resource, adopters should verify whether software versions, API links, datasets, and institutional assessment requirements remain current.

0.5.4 Finding Materials Quickly

The fastest way to use this volume is to combine chapter-level navigation with search. The table of contents identifies the major chapters and sections. The abstract and descriptor elements provide quick overview of each institutional chapters. The index of terms provides another entry point for locating recurring PDC concepts, activity names, tools, assessment terms, and implementation issues across the institutional chapters. The index and PDF search can be used to locate activity names, PDC concepts, tools, and evidence terms.

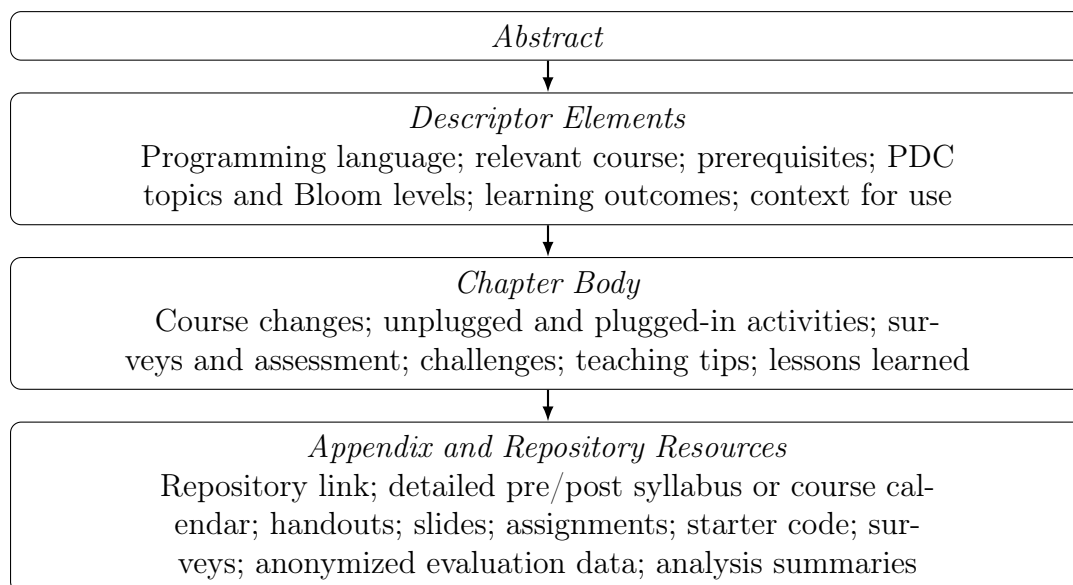


Figure 4: Common structure of institutional exemplar chapters.

Useful search terms include: Flag Maker, Penny Search, Penny Sorting, OpenMP, Earthquake Tracker, Greenfoot, animation, visualization, data parallelism, speedup, load balance, event handling, remote data, and API. Readers looking for assessment material should search for terms such as pre/post survey, student feedback, engagement, assessment, reflection, and data analysis.

0.5.5 Summary Guidance

This volume is best used as a guided collection of adaptable exemplars rather than as a linear book. Begin with an adoption goal, use Chapter 1 to understand the common activity pattern, compare a small number of institutional chapters that resemble the local context, and then move to appendices and repositories for implementation resources. A small, feasible first adoption is often the best starting point for broader CS1 modernization.

Appendix

Prior Work Framing this Project

The NSF-supported Center for Parallel and Distributed Computing Curriculum Development and Educational Resources (CDER)⁴ has worked for more than a decade to integrate suitable parallel and distributed computing concepts into undergraduate computing curricula, with particular emphasis on early and core courses with a premise that all computing graduates have a certain skill set in PDC to modern workforce. The CS1/CS2 exemplar project described in this volume did not arise in isolation. It builds on a sequence of curriculum-development, early-adoption, instructor-training, dissemination, and community-building efforts that together shaped the need for this project and made it feasible. In particular, the project draws on the IEEE TCPP Curriculum Initiative, CDER early adopter programs, the CDER book series, NSF-supported conceptualization and implementation grants, the Edu* workshop series, and training workshops for instructors.

The TCPP Curriculum Initiative: The IEEE Computer Society Technical Committee on Parallel Processing (TCPP) Curriculum Initiative began with a central question in 2010 during which Prasad was its chair: what should every computer science and computer engineering undergraduate know about parallel and distributed computing? The resulting curriculum guidelines identified PDC topics across programming, architecture, algorithms, and cross-cutting areas, and mapped these topics to possible levels of coverage and courses in the undergraduate curriculum. Version I of the TCPP curriculum was released in 2012, and Version II β was released in 2020 with expanded attention to Big Data, energy, distributed computing, and pervasive topics.⁵ The initiative helped make the case that PDC is not only an advanced elective topic, but a body of concepts that should be introduced throughout the undergraduate curriculum, including in early core courses. The ACM/IEEE Computer Science Curricula 2013 report explicitly linked to the TCPP curriculum for more complete PDC coverage, and CS2023 includes a dedicated PDC knowledge area that cites the Version II β effort. This national curriculum framing provided the intellectual foundation for moving from topic guidelines toward concrete course exemplars.

Early Adopters: Curriculum guidelines alone do not change courses; instructors need examples, incentives, community, and evidence. The CDER early adopter competitions were created to incentivize instructors who were willing to incorporate PDC topics into existing undergraduate courses and report on their experiences. Early adopter awards funded by NSF, TCPP, Intel, NVIDIA, and IBM provided modest seed funding and some equipment, but their broader purpose was to create a community of practitioners who could test, adapt, and refine PDC curricular ideas in varied institutional settings. Over multiple competitions, more than 140 early adopters were supported, and the broader CDER community of early adopters and trained instructors has grown to roughly 300 participants or institutions nationally and

⁴<https://cdercenter.org/pdc-center/>

⁵<https://cdercenter.org/pdc-curriculum/>

internationally. These early adopters demonstrated that PDC could be infused into many parts of the curriculum, but they also revealed a persistent need: instructors wanted more complete, coherent, classroom-tested course exemplars, especially for early courses where adoption barriers are highest.

CDER book series: The CDER book series provided another mechanism for collecting, curating, and disseminating PDC teaching materials. The first two volumes, published by Morgan Kaufmann and Springer, gathered instructor- and student-facing chapters on topics such as introductory parallelism, threads, fork-join parallelism, performance, scalability, energy efficiency, MapReduce, GPU computing, mobile concurrency, and integrative GUI applications. These volumes have been made available as free preprint chapters through the CDER website and have collectively reached tens of thousands of chapter downloads. A third CDER volume – expected to be published in late 2026 by Springer – focused on adopter experience, exemplars, and resources, extends this model by emphasizing implementation context and transferability. The present CS1/CS2 exemplar e-book follows the same broad philosophy, but focuses more tightly on modernizing the introductory course sequence through complete course packages, shared activities, assessment instruments, and institutional adaptation narratives, with a vision of creating national models.

Grants and support: Several NSF-supported efforts directly frame this project. The earlier CDER center grant supported the development of PDC curriculum guidelines, dissemination, community building, and early adoption. A subsequent NSF CyberTraining implementation grant supported broader instructor training for incorporating PDC into undergraduate CS and CE curricula. A CyberTraining Institute Conceptualization grant supported stakeholder workshops that examined the systemic barriers to broader PDC adoption, including the continued dominance of a sequential model of computation in early courses, the lack of modern course exemplars, limited instructor preparation, and the difficulty of changing tightly coupled curricular structures. The resulting message was clear: many stakeholders agreed that computing education needed modernization, but potential adopters needed someone to “go first” and show what such modernization could look like in real courses. The NSF Modernizing CS1/CS2 Exemplar Project, Award #2321015, directly addresses that need by developing, transferring, and evaluating PDC-infused introductory course exemplars across diverse institutions.

Edu* workshops and community building: The Edu* workshop series has been a central mechanism for building and sustaining a community of PDC educators. EduPar, held with IPDPS, began as a venue for early adopters and other interested instructors to report experiences, receive feedback, and discuss curricular and pedagogical issues in parallel and distributed computing education. EduHPC, held with SC, created a complementary venue connected to the high-performance computing community, while EduHiPC, held with HiPC in India, extended the model internationally. Together, EduPar, EduHPC, and EduHiPC have provided recurring forums for papers, posters, panels, working meetings, special issues, and community feedback. These workshops have helped transform isolated adoption efforts into an ongoing professional community of instructors, curriculum developers, and PDC

researchers interested in undergraduate education.

Instructor training workshops and related activities: Distinct from the Edu* research and community workshops, CDER has also organized instructor-facing training activities aimed at helping faculty prepare, adapt, implement, and evaluate PDC-infused course materials. These include NSF CyberTraining week-long summer workshops, shorter tutorials and special sessions at SIGCSE, training activities associated with EduHiPC, and hybrid or online training events. Such workshops emphasize hands-on preparation: introducing PDC concepts, walking through classroom activities and programming assignments, helping instructors identify appropriate course insertion points, and supporting assessment plans. This training infrastructure helped prepare early adopters and later this exemplar-project participants to move from interest in PDC education to concrete classroom implementation. All our development and testing team members were past trainees or early adopters.

Concluding remarks

Taken together, these activities created the foundation for the present CS1/CS2 exemplar project. The TCPP curriculum initiative identified what PDC concepts undergraduates should learn; early adopters and training workshops showed both the promise and difficulty of incorporating those concepts into real courses; the CDER book series and courseware repository provided mechanisms for dissemination; and NSF-supported planning and implementation grants clarified the need for transferable, evidence-informed exemplars. The work reported in this volume is therefore the next step in a longer CDER effort: moving from guidelines, isolated modules, and instructor training toward complete, classroom-tested introductory course exemplars that can help departments modernize the computational model students encounter at the beginning of their computing education.